

ПАРАЛЛЕЛИЗМ НА УРОВНЕ ПОТОКОВ

Архитектура процессоров и технология CUDA

Перцев Дмитрий – 04.10.2026

Содержание лекции



Часть I. OpenMP

1. OpenMP и его характеристики
2. Директивы и формат представлений
3. Директива parallel и распараллеливание циклов
4. Локальные переменные и редукция
5. Планирование циклов
6. Структура Sections и барьеры
7. Функции OpenMP API



OPENMP И ЕГО ВОЗМОЖНОСТИ

OpenMP



OpenMP (Open Multi-Processing) — API для многопоточного параллелизма на системах с общей памятью (Shared Memory).

Поддерживаемые языки: C, C++, Fortran

Официальный сайт: www.openmp.org

OpenMP

Состояние компиляторов



Компилятор	Поддержка OpenMP	Примечания
GCC 13/14	Полная поддержка OpenMP 5.2	Рекомендуется для Linux
Clang/LLVM	Отличная поддержка OpenMP 5.x	Включая offloading на GPU
MSVC 2022 (v17.10+)	OpenMP 3.0+ / частичная 5.0	Использовать флаг `/openmp:llvm`
MSVC (старый)	Только OpenMP 2.0	Плана по обновлению нет

Модель выполнения: Fork-Join

Все начинается с главного потока (Master Thread).

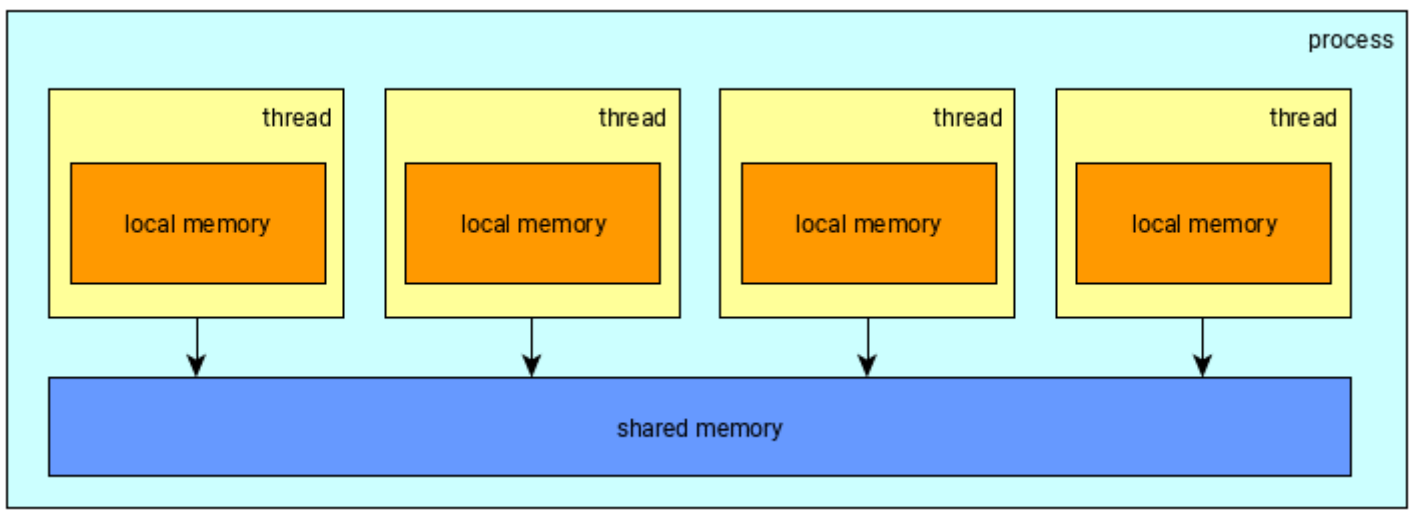


Рис. 1. Модель памяти и выполнения в OpenMP

Принцип работы:



1. **Fork (Вилка):** При встрече директивы `parallel` главный поток создает команду (team) дочерних потоков
2. **Execution:** Код внутри параллельной области выполняется потоками конкурентно
3. **Join (Соединение):** В конце параллельной области дочерние потоки завершаются или приостанавливаются, и управление возвращается главному потоку

Формат представлений директив



```
#pragma omp <директива> [ [ , ] <директива>] ... ]
```

Основные директивы:

- `parallel` — создание параллельной области
- `for` — распараллеливание цикла
- `parallel for` — комбинация (создание области + распараллеливание)
- `section` / `sections` — параллельное выполнение различных блоков кода
- `single` — выполнение блока одним потоком
- `master` — выполнение блока только главным потоком
- `critical` — критическая секция (взаимное исключение)
- `atomic` — атомарная операция
- `ordered` — упорядоченное выполнение
- `flush` — синхронизация памяти



- `task` — создание задачи (OpenMP 3.0+)
- `simd` — векторизация (OpenMP 4.0+)
- `target` — offloading на ускоритель (OpenMP 4.0+)

Директива parallel



Директива parallel

Формат:



```
#pragma omp parallel [ <clause> [ , ] <clause>] ...
```

Структурный блок

Все переменные, объявленные **до** директивы `parallel`, по умолчанию являются **общими** (shared).

Директива parallel

Пример:



```c
#include <omp.h>
#include <stdio.h>

int main() {
    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        printf("Hello from thread %d\n", thread_id);
    }
    return 0;
}
```

Атрибуты данных (Data-Sharing Attributes)



Критически важная тема для понимания многопоточности.

Основные атрибуты:

- `shared(x)` — все потоки работают с одной ячейкой памяти
- `private(x)` — каждый поток создает свою **неинициализированную** локальную копию
- `firstprivate(x)` — каждый поток получает свою копию, **инициализированную** значением из главного потока
- `lastprivate(x)` — по завершении области, значение переменной копируется из потока, который выполнил **последнюю итерацию**
- `default(none)` — **Best Practice!** Запрещает автоматическое определение типов

Атрибуты данных (Data-Sharing Attributes)



Пример:

```
int x = 10;
#pragma omp parallel for default(none) shared(array) firstprivate(x)
for (int i = 0; i < 100; i++) {
    // x здесь равно 10 для каждого потока, но изменения x не влияют на других
    array[i] = x + i;
}
```

Локальные переменные



Локальные переменные

Проблема:



```
int temp;
#pragma omp parallel for
for (i=0; i < 100; i++) {
    temp = array[i];
    array[i] = do_something(temp);
}
```

Все переменные до директивы `parallel` общие — это race condition!

Локальные переменные

Решение 1: Объявление внутри цикла



```
#pragma omp parallel for
for (i=0; i < 100; i++) {
    int temp;  // Локальная переменная
    temp = array[i];
    array[i] = do_something(temp);
}
```

Локальные переменные

Решение 2: Явное указание `private`



```
int temp;
#pragma omp parallel for private(temp)
for (i=0; i < 100; i++) {
    temp = array[i];
    array[i] = do_something(temp);
}
```

Распараллеливание циклов



```
#pragma omp parallel for
for (i=0; i < numPixels; i++) {
    pGrayScaleBitmap[i] = (unsigned BYTE)
        (pRGBBitmap[i].red   * 0.299 +
         pRGBBitmap[i].green * 0.587 +
         pRGBBitmap[i].blue  * 0.114);
}
```

Редукция



Редукция позволяет безопасно агрегировать данные из локальных копий потоков в одну глобальную переменную, избегая гонок данных (race conditions).

Редукция

Пример:



Редукция

Последовательный код:

```
sum = 0;
for (i=0; i < 100; i++) {
    sum += array[i];
}
```

Редукция

Параллельный код:

```
sum = 0;
#pragma omp parallel for reduction(+ : sum)
for (i=0; i < 100; i++) {
    sum += array[i];
}
```

Редукция

Таблица операторов редукции:

Оператор директивы reduction	Инициализирующее значение (каноническое)
+	0
-	0
*	1
&	0 (каждый бит установлен)
^	0
|	0
&&	1
||	0
min	Максимальное значение типа
max	Минимальное значение типа

Планирование циклов



Как итерации цикла распределяются между потоками.

```
#pragma omp parallel for schedule(kind [, chunk size])
```

Основные режимы планирования:

- **`static`** — итерации делятся поровну заранее. Минимум накладных расходов, но возможна дисбалансировка нагрузки
- **`static, chunk`** — итерации делятся блоками указанного размера
- **`dynamic`** — потоки берут по одной (или `chunk`) итерации по мере освобождения. Отлично для циклов с неравномерной сложностью
- **`guided`** — размер чанка экспоненциально уменьшается. Компромисс между static и dynamic
- **`auto`** — решение принимает компилятор



- **`runtime`** — решение принимается в момент выполнения на основе переменной окружения `OMP_SCHEDULE`

Планирование циклов

Примеры:



```
// Статическое планирование
#pragma omp parallel for schedule(static)
for (i=0; i<100; i++) { ... }

// Динамическое планирование с размером блока 10
#pragma omp parallel for schedule(dynamic, 10)
for (i=0; i<100; i++) { ... }

// Guided планирование
#pragma omp parallel for schedule(guided, 5)
for (i=0; i<100; i++) { ... }
```

Вложенные циклы и `collapse`



Если у вас вложенный цикл, а число итераций внешнего цикла меньше числа ядер, используйте `collapse(n)`, чтобы «схлопнуть» их в один большой одномерный цикл для лучшего распределения.

Вложенные циклы и `collapse`

Пример:



```
#pragma omp parallel for collapse(2)
for(int i=0; i<N; i++)
    for(int j=0; j<M; j++)
        matrix[i][j] = compute(i, j);
```

Структура Sections



Позволяет выполнять различные блоки кода параллельно.

```
#pragma omp parallel
{
    #pragma omp for
    for (i=0; i<x; i++)
        fn1();

    #pragma omp sections
    {
        #pragma omp section
        {
            TaskA();
        }

        #pragma omp section
        {
            TaskB();
        }
```

```
        #pragma omp section
        {
            TaskC();
        }
    }
}
```

Барьеры и Операция Nowait



Барьер — точка синхронизации, в которой все потоки должны дождаться друг друга. Неявный барьер стоит в конце каждой `parallel` области.

Барьеры и Операция Nowait

Явный барьер:



```
#pragma omp parallel {
    // A bunch of work...

    #pragma omp barrier
    // Все потоки ждут здесь

    // Additional work...
}
```

Барьеры и Операция Nowait

Nowait (отмена неявного барьера):



```
#pragma omp parallel
{
    #pragma omp for nowait
    for (i=0; i<100; i++)
        compute_something();

    // Нет барьера! Потоки сразу переходят дальше

    #pragma omp for
    for (j=0; j<500; j++)
        more_computations();
}
```

Главный поток и Дополнительные потоки



```
#pragma omp parallel
{
    do_multiply_times();

    #pragma omp for
    for (i=0; i<100; i++)
        fn1();

    #pragma omp master
    fn_for_master_only();  // Выполняется только главным потоком (thread 0)

    #pragma omp for nowait
    for (i=0; i<100; i++)
        fn2();

    #pragma omp single
    one_time_any_thread();  // Выполняется любым одним потоком (не обязательно главным)
}
```

Синхронизация и Защита данных



Синхронизация и Защита данных

`critical` — критическая секция (мьютекс)



Только один поток может выполнять блок в единицу времени.

```
#pragma omp critical
{
    shared_resource.update();
}
```

Синхронизация и Защита данных

`atomic` — атомарная операция



Быстрее `critical`, но работает только с простыми выражениями.

```
#pragma omp atomic update
counter++;

#pragma omp atomic read
x = shared_var;

#pragma omp atomic write
shared_var = x;
```

Синхронизация и Защита данных

`ordered` — упорядоченное выполнение



Гарантирует выполнение итераций цикла в строгом последовательном порядке.

Сильно бьет по производительности!

```
#pragma omp parallel for ordered
for (i=0; i<100; i++) {
    // Параллельные вычисления
    compute(i);

    #pragma omp ordered
    {
        // Эта часть выполняется строго по порядку
        printf("%d\n", i);
    }
}
```

Задачи (Tasks) — OpenMP 3.0+



Главное отличие современного OpenMP от простого распараллеливания циклов.

Позволяет распараллеливать **неструктурированный код**, рекурсию, обход деревьев и графов.

Задачи (Tasks) — OpenMP 3.0+

Пример: Обход дерева



```
void traverse_tree(Node* node) {
    if (node == NULL) return;

    // Обработка текущего узла
    process_node(node);

    #pragma omp task
    traverse_tree(node->left);

    #pragma omp task
    traverse_tree(node->right);
}

int main() {
    #pragma omp parallel
    {
        #pragma omp single
        {
            traverse_tree(root);
        }
```

```
    }
    return 0;
}
```

Задачи (Tasks) — OpenMP 3.0+

`taskwait` — ожидание завершения задач



```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        process_node(tree->left);

        #pragma omp task
        process_node(tree->right);

        #pragma omp taskwait  // Ожидание завершения всех дочерних задач
    }
}
```

SIMD-векторизация — OpenMP 4.0+



Позволяет явно указать компилятору, что итерации цикла независимы и их можно выполнить векторно (загрузить несколько значений в SIMD-регистры процессора).

SIMD-векторизация — OpenMP 4.0+

Пример:

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

SIMD-векторизация — OpenMP 4.0+

SIMD с редукцией:



```
float sum = 0.0f;
#pragma omp simd reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += array[i];
}
```

SIMD-векторизация — OpenMP 4.0+

Векторизация функций:



```
#pragma omp declare simd
float compute(float x, float y) {
    return x * x + y * y;
}

#pragma omp simd
for (int i = 0; i < N; i++) {
    result[i] = compute(A[i], B[i]);
}
```

Гетерогенные вычисления (Offloading) — OpenMP 4.0+



Перенос вычислений на ускорители (GPU, Xeon Phi) с использованием единого исходного кода.

Гетерогенные вычисления (Offloading) — OpenMP 4.0+



Пример:

```
double A[N], B[N], C[N];
// ... инициализация ...

#pragma omp target teams distribute parallel for \
    map(to: A, B) map(from: C)
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

Основные директивы:

- `target` — перенос данных и кода на устройство
- `teams` — создание команд на GPU (аналог Thread Blocks в CUDA)
- `distribute` — распределение итераций между командами GPU
- `map(to: ...)` — копирование данных на устройство
- `map(from: ...)` — копирование данных с устройства



- `map(tofrom: ...)` — копирование в обе стороны

Runtime API



Runtime API

Полезные функции:



```c
// Возвращает количество потоков в текущей команде
int omp_get_num_threads(void);

// Устанавливает количество потоков (переопределяет OMP_NUM_THREADS)
int omp_set_num_threads(int NumThreads);

// Возвращает номер текущего потока (0 - master thread)
int omp_get_thread_num(void);

// Возвращает количество доступных процессоров
int omp_get_num_procs(void);

// Возвращает время в секундах (высокого разрешения) - ИДЕАЛЬНО ДЛЯ ЗАМЕРОВ!
double omp_get_wtime(void);

// Разрешает/запрещает динамическое изменение числа потоков
void omp_set_dynamic(int dynamic_threads);
```

Runtime API

Пример замера времени:



```
double start_time = omp_get_wtime();

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    compute(i);
}

double end_time = omp_get_wtime();
printf("Time: %f seconds\n", end_time - start_time);
```

Переменные окружения



Настраиваются **до** запуска программы.

Переменная	Описание
`OMP_NUM_THREADS=8`	Установка числа потоков
`OMP_SCHEDULE="dynamic, 10"`	Планирование по умолчанию
`OMP_PROC_BIND=close`	Привязка потоков к ядрам CPU (критично для производительности!)
`OMP_PLACES=cores`	Распределение потоков по физическим ядрам
`OMP_DISPLAY_ENV=TRUE`	Вывод информации о среде OpenMP при запуске

Переменные окружения

Пример использования (Linux):



```
export OMP_NUM_THREADS=8
export OMP_PROC_BIND=close
export OMP_PLACES=cores
./my_program
```

Подводные камни и Best Practices



Подводные камни и Best Practices

1. False Sharing (Ложный кэш-промах)



Если два потока пишут в разные переменные, но они находятся в одной кэш-линии (обычно 64 байта), ядра процессора будут постоянно инвалидировать кэш друг друга.

Решение: Выравнивание данных (padding) или использование `private` массивов с последующей редукцией.

```
// Плохо:
int counters[NUM_THREADS];  // Могут попасть в одну кэш-линию

// Хорошо:
struct alignas(64) PaddedCounter {
    int value;
    char padding[60];  // Заполнение до 64 байт
};
PaddedCounter counters[NUM_THREADS];
```

Подводные камни и Best Practices

2. Накладные расходы на Fork-Join



Создание потоков — дорогая операция. Нельзя заходить в `#pragma omp parallel` внутри глубоко вложенных циклов или часто вызываемых функций. Параллельная область должна быть «снаружи».

Подводные камни и Best Practices

3. Дисбалансировка нагрузки (Load Imbalance)



Использование `schedule(static)` для циклов, где сложность итераций растет. **Решение:** `schedule(dynamic)`.

Подводные камни и Best Practices

4. Использование `default(none)`



Всегда явно указывайте атрибуты данных, чтобы избежать скрытых гонок данных.

```
#pragma omp parallel for default(none) shared(array, N) private(temp)
for (int i = 0; i < N; i++) {
    // ...
}
```

Компиляция



Компиляция

GCC:

```
g++ -fopenmp program.cpp -o program
```

Компиляция

Clang:



```
clang++ -fopenmp program.cpp -o program
```

Компиляция

MSVC (VS 2022 v17.10+):

```
cl /openmp:llvm program.cpp
```

Компиляция

MSVC (старый, только OpenMP 2.0):

```
cl /openmp program.cpp
```

Полезная литература и ресурсы



Полезная литература и ресурсы

Официальная документация:



- OpenMP API Specification 5.2
- PDF спецификация

Полезная литература и ресурсы

Книги:



- **Michael McCool, Arch Robison, James Reinders** — «Structured Parallel Programming: Patterns for Efficient Computation»
- **Evgueni Petrov** — «OpenMP 5.0 for C/C++ and Fortran»
- **Barbara Chapman, Gabriele Jost, Ruud van der Pas** — «Using OpenMP»

Полезная литература и ресурсы

Инструменты профилирования:



- **Intel VTune Profiler** / **AMD uProf** — для анализа False Sharing и эффективности векторизации
- **Perf** (Linux) — базовый анализ кэш-промахов
- **Valgrind (Helgrind)** — поиск гонок данных

Полезная литература и ресурсы

Статьи и ресурсы:



- Шпаргалка по OpenMP
- Разбираемся, не так просто, как кажется
- Официальный сайт OpenMP